

DETAILED & DESCRIPTIVE INTERVIEW PREPARATION

Qualcomm Security Hardware DV Engineer

Comprehensive Answers for Deep Study

GENERAL BEHAVIORAL QUESTIONS

Tell me about yourself - a natural 60-second introduction

I'm Keerthivasan P, based in Cork, Ireland. I have over four years of hands-on experience in embedded systems and ASIC verification, having worked across two primary roles that shaped my current expertise.

My career started at Infosys, where I spent approximately two years as an embedded firmware engineer. There, I developed deep expertise in real-time operating systems, specifically FreeRTOS running on ARM Cortex-M microcontrollers. This wasn't just about writing code—I learned the critical importance of understanding the underlying hardware, interrupt latency, task scheduling, and the subtle interactions between software and silicon. I shipped production firmware to customers, which taught me that quality and reliability aren't negotiable.

After Infosys, I transitioned to ISD Integrated Systems Development, where I spent two years in ASIC validation and protocol verification. This was a deliberate career shift toward verification, where I recognized my real passion lay. I built Python-based protocol validation frameworks, worked on SystemVerilog testbenches, and learned how to systematically think about design verification at scale. During this time, I also took on a leadership role redesigning our verification framework, which accelerated protocol onboarding by 3x.

Simultaneously, I pursued a Master's degree in Electrical and Electronics Engineering at UCC. My thesis focused on extending the SERV RISC-V processor with stochastic computing extensions. This project was transformative because it combined hardware design (SystemVerilog RTL), comprehensive verification (UVM testbenches), gate-level simulation, and power characterization—essentially giving me a 360-degree view of the verification landscape.

My core strengths are verification rigor (I don't just test designs; I understand them deeply), systematic problem-solving (breaking complex problems into manageable modules), and learning velocity (the ability to pick up new domains quickly, as evidenced by transitioning from firmware to verification to hardware design).

I'm genuinely excited about the Security Hardware Design Verification Engineer role at

Qualcomm because security IP verification aligns perfectly with where I want to specialize, and I'm committed to Cork as my long-term base.

Why are you interested in the Qualcomm Security Hardware DV role specifically?

There are several reasons, and I want to be specific rather than generic.

First, the Cork location. I'm based here and committed to Cork long-term. I'm not looking for a temporary relocation or using Qualcomm as a stepping stone. I value the quality of life here, and I've built professional relationships in the Cork tech community. For Qualcomm, I represent stability and commitment to grow the Cork office over years, not months.

Second, security IP verification is where I want to specialize. While my background spans firmware, protocol validation, and CPU design, I've increasingly focused on understanding security concepts. My AES-128 design and verification project wasn't random—I deliberately chose cryptographic hardware as a learning domain. I studied the FIPS 197 specification deeply, implemented the algorithm in SystemVerilog for my reference model, and explored concepts like Galois field arithmetic and constant-time implementation. This wasn't for a course; it was intentional preparation for security work.

Third, Qualcomm's standards demonstrate a quality bar that attracts rigorous engineers. The job description mentions AVA_VAN.5 and EAL5+ certification standards. These are formal assurance standards, not just marketing claims. AVA_VAN.5 specifically requires formal specification and architectural verification. EAL5+ is Common Criteria certification typically required for government and financial systems. Companies that pursue these standards don't compromise on verification quality. That appeals to me deeply because I've come to believe that quality and rigor are non-negotiable in verification, especially for security.

Fourth, the role description emphasizes collaboration with architects, SoC designers, and security penetration testers. This suggests Qualcomm treats verification as a multi-dimensional discipline, not just test automation. My experience designing SERV (understanding CPU architecture deeply) and my protocol validation background position me well for this collaborative approach.

Finally, the impact. Security IP in Snapdragon SoCs is used in billions of devices globally. The verification work directly impacts the security of those devices. That's meaningful work—not just shipping a feature, but potentially protecting billions of users. That's what drew me to this role.

Where do you see yourself in five years professionally?

I see myself as a senior or staff-level verification engineer, potentially in an architectural verification leadership role.

To be more specific: In five years, I want to have deepened my expertise in security hardware verification to the point where I'm regularly making architectural decisions, not just testing designs handed to me. I want to be the engineer in the room who can say, 'This architectural choice has these verification implications, and here's how we should approach it.' That requires understanding not just how to verify, but how design decisions impact verifiability and security properties.

I also want to lead complex verification initiatives. Right now, my experience is primarily with subsystems—SERV is a CPU core, AES is a crypto accelerator. At Qualcomm, I'll be part of verifying entire security subsystems that might include multiple accelerators, key management, root of trust, and integration with the broader SoC. I want to own verification strategy for such subsystems, making decisions about UVM architecture, coverage metrics, formal verification targets, and sign-off criteria.

Mentorship is another key part. As I grow technically, I want to mentor junior verification engineers. I've done informal mentoring—at ISD, I mentored the next hire on our new validation framework—and I found it deeply rewarding. In five years, I'd like to have a structured mentorship role, helping the next generation of verification engineers develop the same rigor and systematic thinking I've valued.

I also want to contribute to Qualcomm's verification methodologies and best practices. Not every verification problem is solved; there are always emerging challenges (new cryptographic algorithms, new threat models, scaling verification to larger SoCs). I want to be part of solving those challenges, potentially developing new methodologies or tools that other teams can use.

Finally, I see potential for technical leadership—not necessarily management, but architectural and technical leadership. Being a senior/staff engineer means being consulted on major decisions, setting the technical direction, and having influence across projects. That's the trajectory I'm aiming for.

What are your three biggest strengths as an engineer?

I've thought about this carefully, and I've identified three strengths that consistently show up in my work:

****Technical Depth Over Surface Knowledge****: I don't just verify designs; I understand the micro-architecture deeply. When I worked on SERV, I didn't treat it as a black box to test. I studied the bit-serial architecture, understood why each module was designed the way it was, learned the trade-offs (area efficiency vs. latency), and designed my testbenches based on that understanding. When I designed AES-128, I didn't just implement the algorithm—I studied Galois field arithmetic to understand why MixColumns works, researched Boyar-Peralta S-box optimization to understand the logic structure, and designed my reference model to match the specification exactly. This deep understanding makes me

effective at finding bugs that surface-level testing would miss. At Qualcomm, this means I can understand security architecture deeply, ask the right questions about implementation choices, and design verification approaches that actually catch issues.

****Methodical Problem-Solving Through Decomposition****: I approach complex problems by breaking them into modules, understanding each module's responsibilities, then understanding how they interact. When GLS diverged from RTL in my SERV work, I didn't panic or randomly try fixes. I systematically reduced the test case, compared waveforms cycle-by-cycle, identified the specific signal that diverged first, traced that back to the root cause (SDF pessimism on a timing path), and worked methodically to fix it. This same approach applies to verification challenges—break down the verification problem into tractable pieces, own each piece rigorously, then integrate. This is exactly what UVM enforces (driver, monitor, sequencer, scoreboard all separate concerns), and I've internalized this thinking.

****Learning Velocity and Adaptability****: I've successfully transitioned from firmware engineer to verification engineer to hardware designer. Each domain required learning new tools, new concepts, new ways of thinking. With firmware, it was understanding RTOS scheduling and interrupt handling. With verification, it was UVM, SystemVerilog, coverage metrics. With hardware design, it was RTL thinking, synthesis constraints, timing analysis. I don't just learn surface-level facts; I ingest the reasoning behind approaches. This is crucial because security verification is a specialized domain—I haven't worked extensively with side-channel analysis or formal verification for information-flow properties yet. But my track record shows I can learn these domains rapidly and deeply. At Qualcomm, I can ramp up on whatever specialized knowledge the security IP verification requires.

What area do you want to develop or improve as an engineer?

I want to be direct and honest here: cryptographic algorithms and side-channel analysis are areas where I have foundational knowledge but not yet specialized expertise, and I'm committed to closing that gap.

Specifically, I understand the basic concepts. I've studied AES, understand S-boxes, Galois field arithmetic, and key expansion. I know what side-channel attacks are conceptually—power analysis, timing analysis, electromagnetic emissions. But I haven't done hands-on work designing cryptographic accelerators, evaluating side-channel resistance through power simulation, or implementing countermeasures like masking or constant-time code.

Here's what I've done proactively to address this: I deliberately chose to implement and verify AES-128 as a project. I didn't just code it; I studied FIPS 197 in detail, researched the Boyar-Peralta optimization, implemented a bit-exact reference model, and designed a comprehensive UVM testbench. More recently, I've been reading about side-channel attacks—DPA (Differential Power Analysis), timing attacks, and fault injection. I've explored

how hardware design choices impact side-channel resistance. I understand at least at a theoretical level what we should be checking for.

But I acknowledge this isn't the same as designing and verifying production security IP. That's where I want to grow. I'm not approaching this as a weakness to hide; I'm approaching it as a specialization to develop. The fact that I've already taken initiative to study these areas, chosen projects specifically in this domain, and am genuinely excited to deepen this expertise—I think that demonstrates the right orientation.

At Qualcomm, working on actual security IP verification would accelerate this learning enormously. I'd be working with experts in cryptography and security, verifying designs against actual specifications and standards, and building real expertise. I'm committed to being a sponge during that learning process—asking questions, studying code, understanding the reasoning behind design and verification choices.

Describe a time you failed or made a mistake, and what you learned from it.

Early in my Infosys role, I made an incorrect architectural assumption that led to a system failure, and that experience fundamentally changed how I approach verification.

I was designing real-time control algorithms for an embedded system using FreeRTOS. I made an assumption about interrupt latency—the time between when an interrupt occurs and when the interrupt handler actually executes. I knew FreeRTOS had various interrupt priorities and that higher-priority interrupts could preempt lower-priority ones, but I didn't actually measure or verify the latency for my specific configuration. I assumed based on general knowledge that interrupt latency would be roughly 5-10 microseconds.

I designed my control algorithm assuming this latency. During integration testing on actual hardware, the system started misbehaving. Sensor readings were inconsistent, control outputs were delayed, and the system wasn't responding predictably. For a control system, this is serious. Initially, I blamed the hardware, the OS, the sensors—anything but my assumptions.

I eventually took a systematic approach: I wrote code to actually measure interrupt latency on the real system. I toggled a GPIO pin when the interrupt occurred, toggled it again in the interrupt handler, and measured the time difference. What I found: interrupt latency was approximately 25 microseconds under my specific configuration—more than double my assumption. The source: I hadn't set interrupt priorities correctly in the NVIC (Nested Vectored Interrupt Controller). Higher-priority system tasks were blocking my interrupt.

The technical fix was straightforward: adjust NVIC priority configuration. But the deeper lesson was profound: never assume architectural behavior. Always measure. Always verify. Always understand the underlying hardware and software interactions.

This experience is why, in my SERV verification work, I insisted on understanding every module deeply, building comprehensive testbenches, and validating my assumptions through GLS. In my AES verification, it's why I built a reference model and validated every round against it. It's why I'm committed to understanding security hardware deeply before designing verification approaches.

The lesson directly applies to verification: we're trusted to ensure designs work correctly. We can't fulfill that trust through surface-level testing or unfounded assumptions. We have to dig deep, understand systems thoroughly, and validate every claim. That experience made me a better engineer.

Tell me about a time you took leadership or initiative on a project.

At ISD, I identified an opportunity to improve our verification infrastructure and led a redesign effort that became a case study in how to balance technical vision with pragmatic execution.

The context: ISD was growing, onboarding new protocol verification projects frequently. Our existing validation framework had evolved organically over years without a comprehensive redesign. It was becoming increasingly difficult to extend for new protocols. Engineers spent more time fighting the framework than writing tests. New hires spent weeks understanding the framework before contributing. I saw the pain points directly because, as a junior engineer, I was spending that time.

I approached my manager with a proposal: redesign the framework. The response was cautious—we had active projects with deadlines. But my manager recognized the long-term problem and gave me time to explore options.

I took a systematic approach: First, I analyzed the existing codebase to identify pain points. What made extension difficult? What required boilerplate code? What discouraged reuse? Second, I researched better patterns. I studied how other teams approached protocol validation, explored Python class hierarchies for test organization, and designed a data-driven configuration approach that could abstract protocol differences.

Third, I created a prototype. I didn't write a 100-page design document; I built a working example for a simple protocol. This let me validate my ideas and get real feedback from senior engineers. Fourth, I iterated based on feedback and gradually migrated existing protocols to the new framework.

The results: The new framework reduced protocol onboarding from weeks to days. Manual test writing decreased 40%. Code reuse increased dramatically. Engineers could express test intent more clearly. Within six months, it became the standard approach for new projects.

But here's what I'm proud of beyond the numbers: I didn't impose the redesign top-down. I

built consensus. I mentored the next hire on the framework, making sure it was maintainable by the team, not just by me. I documented my reasoning and design choices so others could understand the architecture. I created something that enhanced the team's capability, not just impressed management.

This experience taught me that engineering leadership isn't about having the loudest voice or the most authority. It's about seeing problems, proposing solutions, building proof-of-concept, getting consensus, executing systematically, and mentoring others to continue the work. That's the kind of leadership I want to practice at Qualcomm.

SERV RISC-V PROCESSOR - DETAILED PROJECT EXPLANATION

Walk me through your SERV RISC-V processor architecture and design in detail

SERV is a fascinating design because it inverts the typical processor architecture. Instead of a traditional 32-bit wide datapath where all 32 bits are processed in parallel, SERV uses a 1-bit wide datapath where all operations are serialized across 32 cycles.

****High-Level Architecture****: The core philosophy is: sacrifice latency (32 cycles per instruction instead of 1-3 cycles like modern processors) to gain dramatic area and power efficiency. For embedded systems, IoT devices, and security chips where clock cycles are cheap but silicon area is expensive, this trade-off is attractive.

****Key Modules and Their Roles****:

1. ****serv_top****: This is the top-level module that instantiates all sub-blocks. It manages the overall dataflow, defines parameters (W for word width, initially 1 for standard SERV, but can be parameterized for different widths), and connects all major components. The interface includes instruction memory (Wishbone bus), data memory, register file, and CSR (control/status registers) for interrupts and traps.
2. ****serv_decode****: Instruction decoding logic. Takes the 32-bit instruction and generates control signals for all other modules. For example, if the instruction is ADD, it signals the ALU to add; if it's LW, it signals the memory interface to load. This module also performs immediate extraction—pulling out the immediate field from the instruction.
3. ****serv_ctrl****: Program counter (PC) management. In a traditional processor, the PC is 32-bit wide and updated every cycle. In SERV, the PC is still 32-bit but updated over multiple cycles. **serv_ctrl** handles PC increment for sequential execution, branch/jump target calculation, trap handler addresses (for exceptions), and return addresses (for function calls).
4. ****serv_alu****: The arithmetic-logic unit. Despite being 1-bit wide, it performs full 32-bit operations. For example, a 32-bit addition happens over 32 cycles, with the ALU adding bit 0

in cycle 1, bit 1 in cycle 2, etc. This is where the real magic happens—the shifter network that creates a carry propagation chain across 32 cycles.

5. ****serv_rf_top/rf_if****: Register file interface. Manages reading from and writing to the 32 RISC-V general-purpose registers (plus optional CSRs for privileged operations). The interface has dual read ports (RS1 and RS2) and a write port (RD), allowing one register write and two register reads per instruction (sequentially across cycles).

6. ****serv_rf_ram****: The actual register file storage. Typically implemented as a single-port RAM, where reads and writes are muxed carefully to avoid conflicts.

7. ****serv_mem_if****: Memory interface for load/store operations. Handles address calculation, byte selection for unaligned accesses, and load sign-extension (for LB, LH) or zero-extension (for LBU, LHU).

8. ****serv_state****: The state machine that orchestrates everything. For each instruction, this module generates control signals that vary cycle-by-cycle. It tracks which cycle we're on (0-31), knows which control signals should be active at each phase, and manages transitions between instructions.

9. ****serv_csr****: Control/Status Registers for trap handling. Stores MSTATUS (machine status), MIE (machine interrupt enable), MCAUSE (exception cause), MEPC (exception program counter). When an exception occurs (like an interrupt), the CPU saves the current PC and control state to these CSRs.

10. ****serv_bufreg/bufreg2****: Shifter/buffer registers. These are critical for serialization. They stage operands across the 32-cycle instruction window, gradually shifting data in/out.

****Dataflow Example—Adding Two Registers (ADD R1, R2, R3)****:

Cycle 1:

- Decode detects ADD instruction
- RS1 = R2, RS2 = R3 are registered
- ALU is signaled to add

Cycles 1-32:

- Bit 0 of R2 and bit 0 of R3 are read from register file
- ALU adds these two bits (0+0, 0+1, 1+0, or 1+1) plus carry from previous cycle
- Result bit goes into RD (R1) buffer, starting at bit 0
- This repeats for all 32 bits

Cycle 33:

- RD is written to R1 (the destination register)

- Next instruction begins

This serialization is why SERV requires 32 cycles per instruction—every operation, including the most trivial ones, takes the full 32 cycles.

****Parameterization****:

The key parameter is W (word width). In standard SERV, $W=1$ (bit-serial). But the design is parameterized, so you could set $W=4$ to process 4 bits per cycle (requiring 8 cycles per instruction) or $W=32$ for traditional parallel processing. This flexibility was crucial for my thesis work.